Software Architecture and Design Specification (SAD)

Project: Digital Campus Library Reservation Gateway
Version: 1.0
Authors: Dhareen G (SRN: PES1UG24AM084)
Date: 28-10-2026
Status: Draft for submission

Revision history

Version	Date	Author	Change summary	Approval
1.0	28-10-2026	Dhareen G	First complete draft	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Project Guide			

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Digital Campus Library Reservation Gateway (LRG): its components, interfaces, key runtime flows, API design and security architecture.

1.2 Scope

Covers catalog search, authentication, hold management, notifications and librarian administration, as described in the SRS (UC-01 to UC-09). Physical circulation hardware and the internals of campus SSO and the mail service are excluded.

1.3 Audience

Developers, QA engineers, security reviewers, the Head Librarian / library IT, and course evaluators.

1.4 Definitions

LRG (Library Reservation Gateway), SSO, RBAC, REST, JWT (JSON Web Token), ADR (Architecture Decision Record), STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), TLS, DS (Design Specification item).

2. Document overview

2.1 How to use this document

Section 3 gives the architecture (component diagram, pattern, stack, risks, security). Section 4 gives detailed design (sequence diagrams, APIs, error handling, UX). Design item IDs (DS-xxx-nn) in section 3.8 link each requirement in the SRS to the component that satisfies it.

2.2 Related documents

- Software Requirements Specification (SRS) v1.0, including the RTM
- Software Test Plan (STP) v1.0
- Lab 3 architecture justification (component diagram)

3. Architecture

3.1 Goals and constraints

Goals:

- Correct availability and hold queues at all times (no double allocation of a title).
- Fast catalog search (90% of requests within 2 s with 200 concurrent users).
- Secure handling of member identity: SSO only, role-based access, auditable changes.
- Easy to maintain: components with narrow interfaces so a team of students can work in parallel.

Constraints: authentication via campus SSO; email via the institution's SMTP service; open-source tools; one-semester delivery.

3.2 Stakeholders and concerns

Stakeholder	Concerns	Addressed by
Student Members	Fast search, accurate availability, easy holds, privacy	Search component, caching of reads, role/ownership checks

Stakeholder	Concerns	Addressed by
Head Librarian	Accurate data, simple administration, demand insight	Reservation Manager, admin endpoints, demand report
Library IT / Operations	Maintainability, monitoring, uptime	Layered design, health endpoint, structured logs
Institution / Data protection	Personal data protection, audit	SSO only, TLS, audit log, least privilege
Developers	Modularity, clear interfaces	Component interfaces in 3.3 and 4.3

3.3 Component (UML) diagram

The system is split into six components connected through provided (ball) and required (socket) interfaces, and two external systems (campus SSO, email service). The same diagram was submitted in Lab 3.

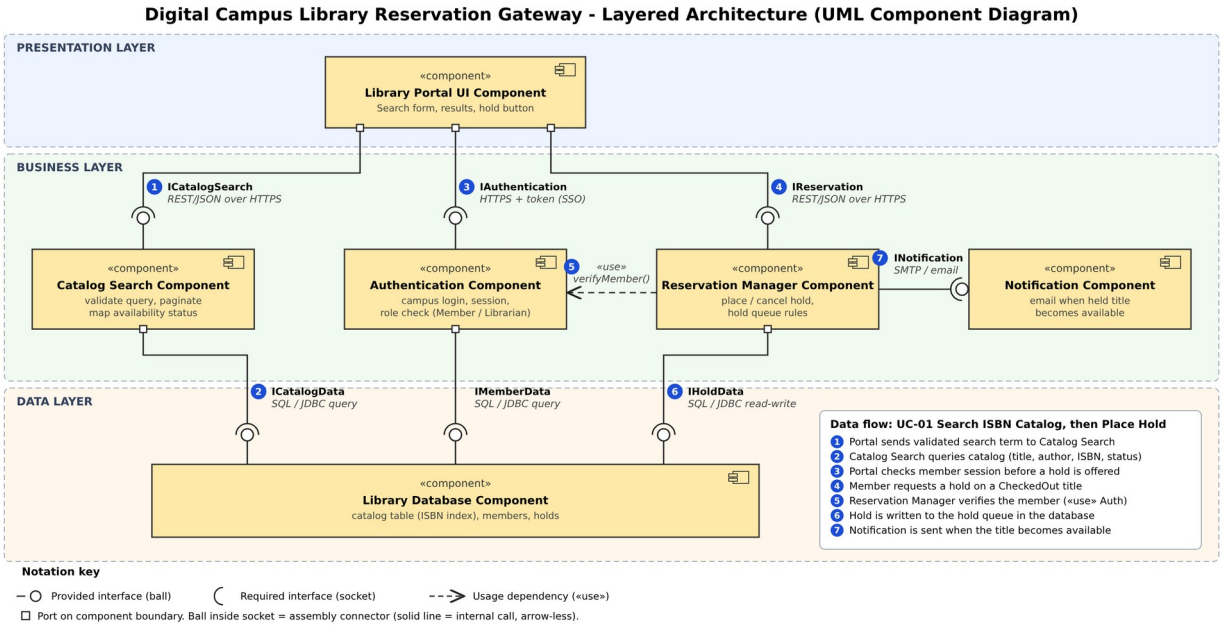


Figure 1: Component diagram of the Library Reservation Gateway

3.4 Component descriptions

Component	Responsibility	Provides	Requires
Library Portal UI	Pages, forms, validation hints, result display, redirects to login	(user-facing web UI)	ICatalogSearch, IAuthentication, IReservation
Catalog Search	Validate queries, search titles by ISBN/title/author, paginate, return book details; admin add/edit of catalog entries	ICatalogSearch	ICatalogData
Authentication	SSO login callback, session and role validation, logout, idle expiry, login throttling	IAuthentication	Campus SSO (external)
Reservation Manager	Place/cancel/list holds, enforce rules (no duplicate, max 5, only CheckedOut), queue positions, mark returned, promote first hold, demand report, audit entries	IReservation	IHoldData, INotification
Notification	Compose and send hold-ready and reminder emails with retry	INotification	Email service (SMTP)
Library Database	Persist titles, members, holds, audit log	ICatalogData, IHoldData	(none)

3.5 Chosen architecture pattern and rationale

A layered architecture (Presentation, Business, Data) deployed as a single modular application was chosen. Components are separate modules with the interfaces above, so each can be tested alone and replaced later.

ADR	Decision	Alternatives rejected and why
ADR-1	Layered modular monolith	Microservices: too much operational overhead and network failure modes for 5,000 members and a student team.
ADR-2	Relational database with transactions	NoSQL: hold queues and uniqueness rules need transactional consistency.
ADR-3	Campus SSO (OpenID Connect) for login	Local passwords: would require storing credentials and a reset flow, increasing risk (SO-1).
ADR-4	Server-side authorization on every request	Client-side hiding of buttons only: bypassable (LRG-SR-003).
ADR-5	Synchronous email send with retry and log	Message queue: unnecessary at this scale; revisit if volume grows.

3.6 Technology stack and data stores

Layer	Choice (proposed)
Front end	HTML, CSS and JavaScript (React or server-rendered templates), responsive
Back end	Java 17 with Spring Boot REST APIs (alternatively Python/Flask)
Data store	PostgreSQL or MySQL with tables: titles, members, holds, audit_log
Security	OpenID Connect SSO, TLS 1.2+, Spring Security, parameterized queries
Email	Institution SMTP over TLS
Testing / tools	JUnit, Selenium, Postman, JMeter, Jira, GitHub

Core data entities: Title(isbn, title, author, status), Member(id, name, email, role), Hold(id, isbn, memberId, state, createdAt, readyUntil), AuditLog(id, userId, action, entity, timestamp). Hold states: Active, Ready, Cancelled, Expired, Fulfilled.

3.7 Risks and mitigations

Risk	Mitigation
Two members' holds race for the same position	Database transaction with unique constraint (isbn, memberId, active) and ordered queue by createdAt
SSO provider outage blocks login	Guests can still search; show a clear message; retry
Email delivery fails or is delayed	Retry 3 times with back-off, log failure, show Ready status in My Holds
Search slows with 100,000 titles	Indexes on isbn, title, author; paging; load test (TC-PERF-01/02)
Librarian data entry errors cause wrong availability	Audit log, confirmation prompt, correction by librarian

3.8 Traceability to requirements

Design item	Requirements	Component(s)
DS-SRCH-01	LRG-F-001 to 005, 042 (part)	Catalog Search; Library Portal UI
DS-AUTH-01	LRG-F-010 to 013	Authentication
DS-RES-01	LRG-F-020 to 024	Reservation Manager
DS-NOT-01	LRG-F-030, 031	Notification
DS-ADM-01	LRG-F-040, 041, 042	Reservation Manager (status, report); Catalog Search (entries)
DS-PERF-01	LRG-NF-001, 005	Indexes, paging, connection pool
DS-DEPLOY-01	LRG-NF-002	Deployment, monitoring, restart policy
DS-SEC-01	LRG-NF-003, LRG-SR-001 to 007	Security architecture 3.9
DS-UX-01	LRG-NF-004	Library Portal UI

Example: R1 (LRG-F-020 Place hold) is realised by the Reservation Manager component (DS-RES-01) using IHoldData and verified by TC-HOLD-01.

3.9 Security architecture

Threat modelling uses STRIDE on the main trust boundaries (browser to portal, portal to SSO, portal to database, portal to mail server).

STRIDE	Threat in LRG	Mitigation	Req
Spoofing	Attacker impersonates a member or the librarian	SSO tokens only, secure cookies, login lockout after 5 failures	LRG-F-010, SR-002, SR-005, SR-006
Tampering	Modify hold or status via forged requests or SQL injection	Server-side authorization, input validation, parameterized queries, TLS	LRG-SR-003, SR-004, SR-001
Repudiation	User or librarian denies placing a hold or changing status	Audit log of user, time, action retained 1 year	LRG-NF-007
Information disclosure	One member reads another's holds; data sniffed in transit	Ownership checks, TLS 1.2+, no sensitive data in logs or errors	LRG-NF-003, SR-001
Denial of service	Search flooding or scraping; brute force	Rate limit 60 req/min/client, login throttling, paging limits	LRG-SR-007, SR-005
Elevation of privilege	Student calls librarian endpoints	RBAC checked in server on every endpoint	LRG-F-012, SR-003

4. Design

4.1 Design overview

The portal UI calls REST endpoints exposed by the business layer. Each request passes an authentication filter (session and role), then the owning component, which uses the data layer. Components never access the database of another component directly; they use the interfaces listed in 3.4.

4.2 UML sequence diagrams

Three flows specific to this project are shown: catalog search (UC-01), placing a hold (UC-03) and returning a title with notification (UC-06, UC-07).

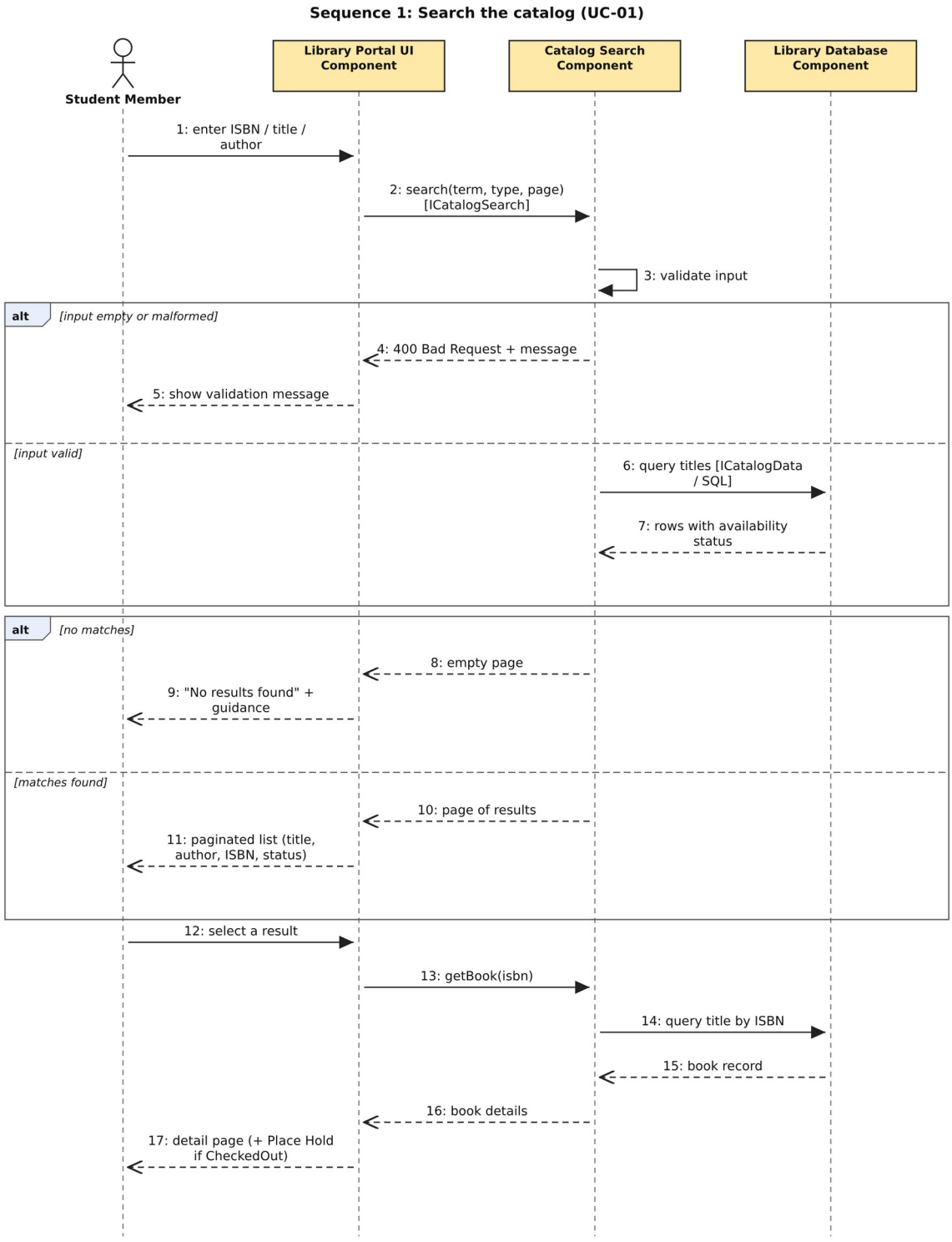


Figure 2: Sequence diagram 1 - Search the catalog (UC-01)

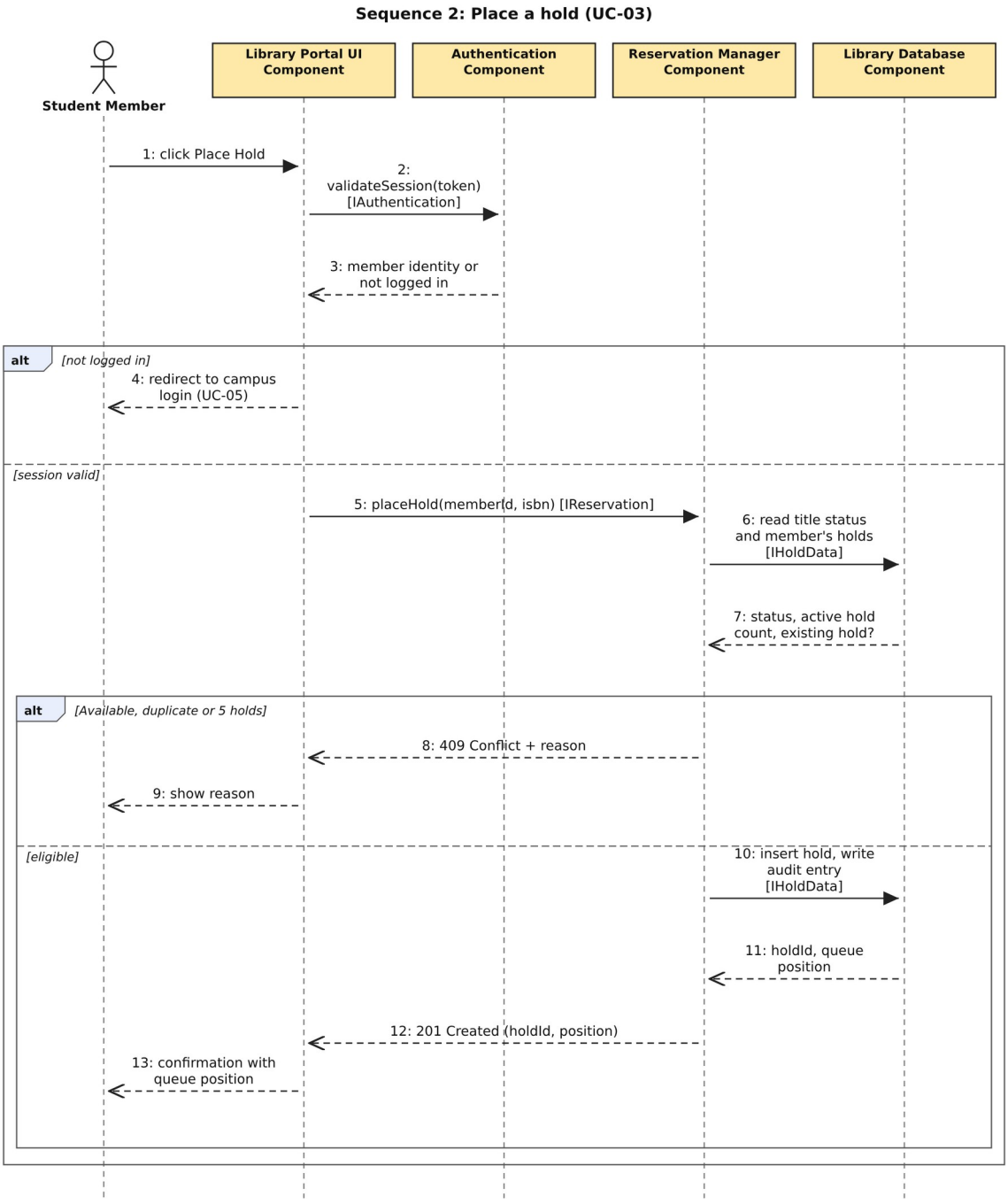


Figure 3: Sequence diagram 2 - Place a hold (UC-03)

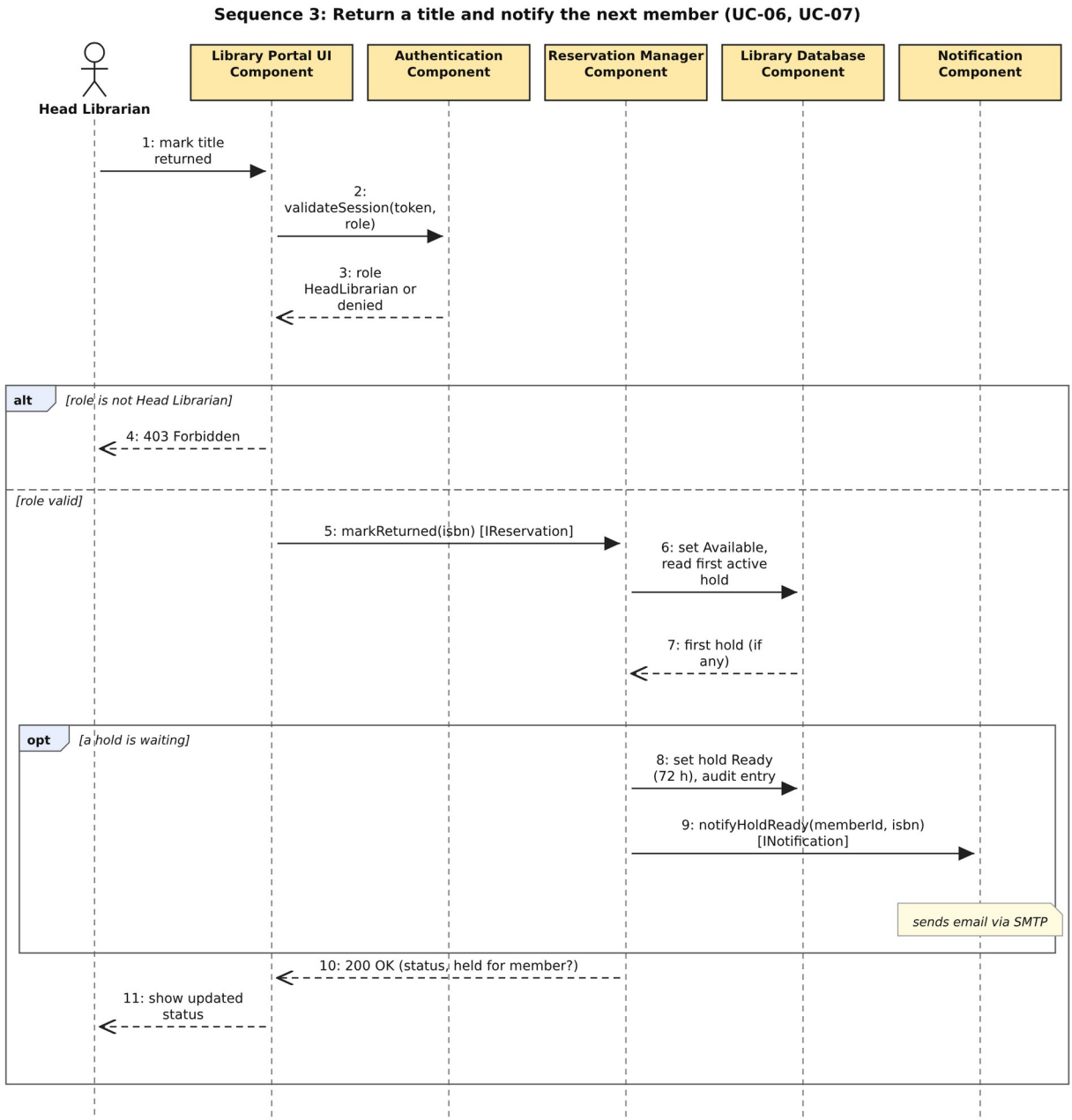


Figure 4: Sequence diagram 3 - Return a title and notify the next member (UC-06, UC-07)

4.3 API design

Interfaces are defined for four components. All endpoints use HTTPS and JSON; authenticated endpoints require a valid session cookie. Error body: {"error": "CODE", "message": "..."}.

4.3.1 Catalog Search component (ICatalogSearch)

Endpoint	Method	Request	Success response	Errors
/api/v1/books	GET	query: q, type (isbn|title|author), page (default 1)	200 {page, pageSize:10, total, results:[{isbn,title,author,status}]}	400 INVALID_QUERY (empty/malformed); 429 RATE_LIMITED
/api/v1/books/{isbn}	GET	path: isbn	200 {isbn,title,author,publisher,year,status,queueLength}	404 NOT_FOUND; 400 INVALID_ISBN
/api/v1/books	POST (librarian)	{isbn,title,author,publisher,year}	201 {isbn}	400; 403 FORBIDDEN; 409 DUPLICATE_ISBN
/api/v1/books/{isbn}	PUT (librarian)	editable fields	200 {isbn}	403; 404

4.3.2 Reservation Manager component (IReservation)

Endpoint	Method	Request	Success response	Errors
/api/v1/holds	POST	{isbn}	201 {holdId, position, state:Active}	401 NOT_LOGGED_IN; 409 TITLE_AVAILABLE | DUPLICATE_HOLD | HOLD_LIMIT; 404
/api/v1/holds/me	GET	(session)	200 [{holdId,isbn,title,state,position,readyUntil}]	401
/api/v1/holds/{id}	DELETE	path: id	204	401; 403 NOT_OWNER; 404
/api/v1/titles/{isbn}/return	POST (librarian)	(none)	200 {status:Available, holdReadyFor: memberId|null}	403 FORBIDDEN; 404; 409 ALREADY_AVAILABLE
/api/v1/reports/demand	GET (librarian)	(none)	200 [{isbn,title,queueLength}] sorted descending	403

4.3.3 Authentication component (IAuthentication)

Endpoint	Method	Request	Success response	Errors
/auth/login	GET	(none)	302 redirect to campus SSO	503 SSO_UNAVAILABLE
/auth/callback	GET	SSO code and state	302 to portal; session cookie set (Secure, HttpOnly, SameSite=Strict)	401 AUTH_FAILED; 429 TOO_MANY_ATTEMPTS
/auth/session	GET	(cookie)	200 {memberId,name,role}	401
/auth/logout	POST	(cookie)	204; session destroyed	-

4.3.4 Notification component (INotification, internal)

Operation	Input	Behaviour	Errors
notifyHoldReady	memberId, isbn	Send email naming the title and the 72-hour pickup deadline; retry 3 times	DELIVERY_FAILED logged; hold stays Ready
sendExpiryReminder	holdId	Send reminder 24 h before expiry unless cancelled	DELIVERY_FAILED logged

4.4 Error handling, logging and monitoring

- Standard error body and HTTP codes (400, 401, 403, 404, 409, 429, 503); messages never reveal stack traces, SQL or other members' data.
- Structured logs with timestamp, user id, action and result; passwords and tokens are never logged (LRG-SR-002).
- Audit log entries for hold create/cancel and every librarian status or catalog change, retained 1 year (LRG-NF-007).
- Monitoring: /health endpoint, uptime check, search response-time percentile, failed-login count, email failure rate.

4.5 UX design

Search box is the main element of the home page. Result rows show a coloured Available / CheckedOut badge and text label (not colour alone). Place Hold appears only for CheckedOut titles and redirects guests to login. My Holds shows queue position and Ready deadline. Responsive layout from 360 px, keyboard navigable, labelled fields, WCAG 2.1 AA contrast (LRG-NF-004).

4.6 Open issues and next steps

- Integration with the library's check-out desk system to update status automatically.
- SMS or push notifications.
- Fine and renewal handling (out of scope for v1.0).
- Confirm the real technology stack with the project team and update 3.6.

5. Appendices

5.1 Glossary

LRG, Hold, Ready, SSO, RBAC, REST, STRIDE, ADR, TLS, WCAG.

5.2 References

IEEE 42010 (architecture description), OWASP Top 10 and ASVS, NIST SP 800-160, UML 2.5, Lab 3 component diagram.

5.3 Tools

Diagrams drawn as SVG, draw.io or PlantUML equivalents; Postman or Swagger/OpenAPI for APIs; Jira and GitHub for tracking and version control.